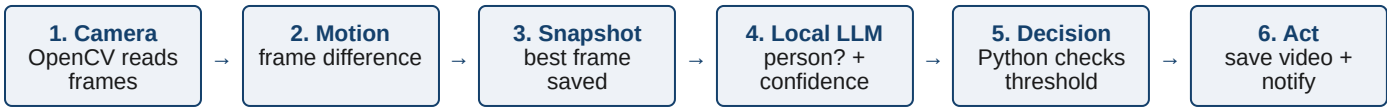
facecam

Camera → motion → local vision LLM → recording + alert · White paper · October 2026

1. Summary

facecam is a Python security-camera system that runs entirely on the owner's own computer. A camera is watched continuously with cheap motion detection. Only when something moves is a single image sent to a **local vision-language model (LLM)**, which says whether a person is in the picture and how sure it is. If a person is confirmed, the Python program saves a video recording and sends a notification to the owner. No image or video leaves the machine except the alert the owner chooses to receive.

2. The whole process



Step	What happens
1. Camera	OpenCV (`cv2.VideoCapture`) reads frames from the webcam or an IP camera. A small rolling buffer keeps the last few seconds in memory, so the recording can include the moments *before* motion was noticed.
2. Motion detection	Each frame is shrunk, blurred and compared with a background model (OpenCV background subtraction). If enough pixels change, motion is declared. This step is very cheap, so it runs on every frame and keeps the LLM idle most of the time. A cooldown (for example 5–10 seconds) stops one event from triggering many LLM calls.
3. Snapshot	When motion starts, the sharpest recent frame is selected and saved as a JPEG, resized to the model's preferred input size to keep inference fast.
4. Local LLM	The image is sent to a vision model running on the same computer (served by Ollama on `localhost`). The prompt asks for strict JSON only: `{"person": true/false, "confidence": 0.0–1.0, "description": "..."}`. The reply is the **accuracy ratio** the system acts on.
5. Decision	Python parses the JSON. If `person` is true and `confidence` is at or above a set threshold (default 0.6), the event is confirmed. Malformed or timed-out replies are treated as "no decision" and logged, never as an alert.
6. Record and notify	On confirmation, Python writes the buffered pre-roll plus live frames to an MP4 (`cv2.VideoWriter`) until motion has stopped for a few seconds. It then sends a notification (phone push via ntfy or Telegram, or email) containing the snapshot, the model's description and its confidence, and the path of the recording.

3. Choosing the local LLM

The goal is the best person-recognition quality with the fastest response, on ordinary hardware. The recommended setup uses two tiers, both small enough to run on a laptop through Ollama:

Role	Model (suggested)	Why
Default	Qwen2.5-VL 3B	Strong at following the "reply in JSON" instruction and at describing people in a scene, at a size that stays responsive on CPU or a modest GPU.
Fastest fallback	Moondream 2	Around 2B parameters, very quick, suited to weak machines; less reliable on small or partly hidden people.
Higher accuracy	Qwen2.5-VL 7B	Better on dark or cluttered scenes if a GPU is available; slower.

This ranking is a starting recommendation, not a measured result. The right choice depends on the user's hardware, so the project should include a small benchmark (section 5) that times each model and scores it on the user's own camera images before one is fixed as the default.

4. Design details that make it reliable

- **Confidence is self-reported.** A language model does not produce a calibrated probability; the number is its own estimate. It is useful for ranking and thresholding but should be tuned against real events, and treated as a guide rather than a guarantee.
- **Two-stage by design.** Motion detection filters out the vast majority of frames, so the slower LLM runs only on a handful of images. This is what makes a local LLM fast enough for real-time use.
- **False alarms.** Pets, shadows, curtains and headlights cause motion without a person. The LLM step exists to reject these; the confidence threshold sets the trade-off between missed people and false alerts.
- **Optional face check.** The existing facecam modules (YuNet detection, SFace recognition) can run on the confirmed snapshot to tell the owner from a stranger, so the owner is not alerted about themselves.
- **Timeouts and fallbacks.** If the LLM does not answer within a few seconds, the system falls back to the faster model or, as a last resort, sends a "motion detected, unverified" alert so an event is never silently lost.
- **Event log.** Every event (time, snapshot, model reply, decision, recording path) is written to a local log for later review and threshold tuning.

5. Proposed code layout

```
facecam/
  camera.py     read frames, rolling pre-roll buffer
  motion.py     background subtraction, cooldown
  vision.py     send snapshot to local LLM, parse JSON reply
  recorder.py   write MP4 clips
  notify.py     ntfy / Telegram / email alerts
  pipeline.py   ties the steps together (main loop)
  cli.py        run, enroll, image, benchmark
  config.py     thresholds, model name, notification settings
```

A `benchmark` command would run each candidate model over a folder of labelled images (person / no person) and report response time and the false-alert and missed-person rates, giving evidence for the model choice and threshold.

6. Privacy, security and limits

- **Local first.** Frames, snapshots, recordings and LLM inference all stay on the machine. The only outbound data is the notification the owner configures; an alert can be sent as text only if the owner prefers.
- **Sensitive data.** Recordings and face data are personal data. Store them on an encrypted disk, restrict access, and delete old clips automatically.
- **Lawful use.** Only film places and people where the law and consent allow it.
- **Not a guarantee.** A vision model can miss a person or be fooled. Treat facecam as an extra layer, not the only protection for anything valuable. Its accuracy has not yet been benchmarked.

7. Status and next steps

Today the repository contains the face detection and recognition modules (enroll, run, image). The camera-to-alert pipeline described here is the design for the next stage. Proposed order of work: (1) motion detection and snapshot, (2) local LLM call with JSON parsing, (3) recording with pre-roll, (4) notifications, (5) benchmark command, (6) optional face check.